

■ BankManager Pro

Système de Gestion Bancaire

Étudiant :	Moustapha Amouchal
Groupe :	ISI-A
Module :	Programmation Java
Établissement :	SUPMTI Rabat
Année :	2025 / 2026
Enseignant :	Pr. Soufiane HAMIDA
Date de remise :	21 Juin 2026

JavaFX 21

SQLite

Maven

MVC

DAO
Pattern

Java 25

1. Introduction

Pour le projet final du module de Programmation Java, j'ai choisi de réaliser un **système de gestion bancaire**. L'idée m'a semblé intéressante parce qu'elle touche à plusieurs aspects à la fois : gérer des clients, des comptes, des transactions. Ce n'est pas juste un simple formulaire CRUD.

Le but était de créer une vraie application de bureau avec une interface graphique agréable, une base de données locale, et une architecture propre. J'ai utilisé **JavaFX** pour l'interface, **SQLite** pour la base de données, et j'ai organisé le code selon le pattern **MVC** avec la couche **DAO**.

L'application s'appelle **BankManager Pro**. Elle permet de gérer des clients, d'ouvrir des comptes bancaires (courant, épargne, crédit), et d'effectuer des opérations financières comme des dépôts, retraits et virements, le tout avec une interface en français.

2. Présentation de l'application

Qu'est-ce que l'application fait ?

En résumé, BankManager Pro est une application de gestion bancaire pour une agence locale. Elle n'est pas connectée à internet, tout tourne en local avec un fichier SQLite. L'interface a quatre sections principales accessibles depuis la barre latérale :

- **Tableau de Bord** : une vue d'ensemble avec des statistiques et un graphique d'historique de solde
- **Gestion des Clients** : ajouter, modifier, rechercher et supprimer des clients
- **Gestion des Comptes** : créer des comptes bancaires liés aux clients, avec filtres
- **Gestion des Transactions** : effectuer des dépôts, retraits et virements avec historique complet

Technologies utilisées

Technologie	Rôle	Version
JavaFX	Interface graphique (UI)	21.0.2
SQLite + JDBC	Base de données locale	3.45.1
Maven	Gestion des dépendances / build	3.9.6
Java	Langage principal	25 (source 21)
JUnit 5	Tests unitaires	5.10.2

Architecture du projet

J'ai organisé le code en suivant le pattern MVC (Modèle, Vue, Contrôleur), combiné avec le pattern DAO pour l'accès aux données. Voici la structure des dossiers :

```
src/main/java/com/banking/
  BankingApplication.java <-- point d'entrée
  model/ <-- Client, Account, Transaction (+ enums)
  dao/ <-- DatabaseManager, ClientDAO, AccountDAO, TransactionDAO
  controller/<-- MainController, DashboardController, ...
  util/ <-- AlertHelper, FormValidator, CurrencyFormatter
src/main/resources/com/banking/
  fxml/ <-- main.fxml, dashboard.fxml, ...
  css/ <-- banking.css
  database/ <-- schema.sql
```

Chaque couche a un rôle précis : les **models** représentent les données, les **DAOs** s'occupent des requêtes SQL, et les **controllers** font le lien entre l'interface et les données. Cette séparation rend le code plus facile à maintenir.

3. Base de données

J'ai utilisé SQLite parce que c'est simple à intégrer dans un projet Java, pas besoin d'installer un serveur. La base de données est un seul fichier **banking.db** qui se crée automatiquement au premier lancement.

Les trois tables principales

Table	Colonnes principales	Description
clients	id, first_name, last_name, email, phone, address, birth_date	Informations personnelles des clients
accounts	id, account_number, client_id, account_type, balance	Comptes bancaires liés à un client (CHEQUE, SAVING, etc.)
transactions	id, source_account_id, target_account_id, transaction_type, amount, timestamp	Historique de toutes les opérations financières

Les tables sont reliées par des clés étrangères : un compte appartient à un client, et une transaction fait référence à un ou deux comptes (source et/ou cible). J'ai aussi activé **PRAGMA foreign_keys = ON** pour que SQLite vérifie ces contraintes automatiquement.

■ **Données de test** : La base est pré-remplie avec 10 clients, 16 comptes et 30 transactions au premier lancement. Les données incluent des noms français (Alice Martin, Bob Dupont...) et marocains (Mohamed Al-Omari, Fatima Benali, Youssef Idrissi, Amina Ziani...).

4. Fonctionnalités principales

4.1 Tableau de Bord

La première page qu'on voit en ouvrant l'application, c'est le tableau de bord. Il affiche quatre cartes statistiques en haut : le nombre total de clients, le nombre de comptes actifs, le total des actifs gérés, et le volume des transactions du mois en cours.

En bas, il y a deux zones : à gauche un **graphique linéaire** qui montre l'historique du solde d'un compte sélectionné (on peut changer le compte via le menu déroulant), et à droite un tableau des **10 dernières transactions**.

Le graphique est ce que j'ai ajouté comme fonctionnalité originale. Il reconstruit l'historique du solde en rejouant toutes les transactions dans l'ordre chronologique. Pas de colonne historique stockée dans la DB, juste un calcul à la volée.

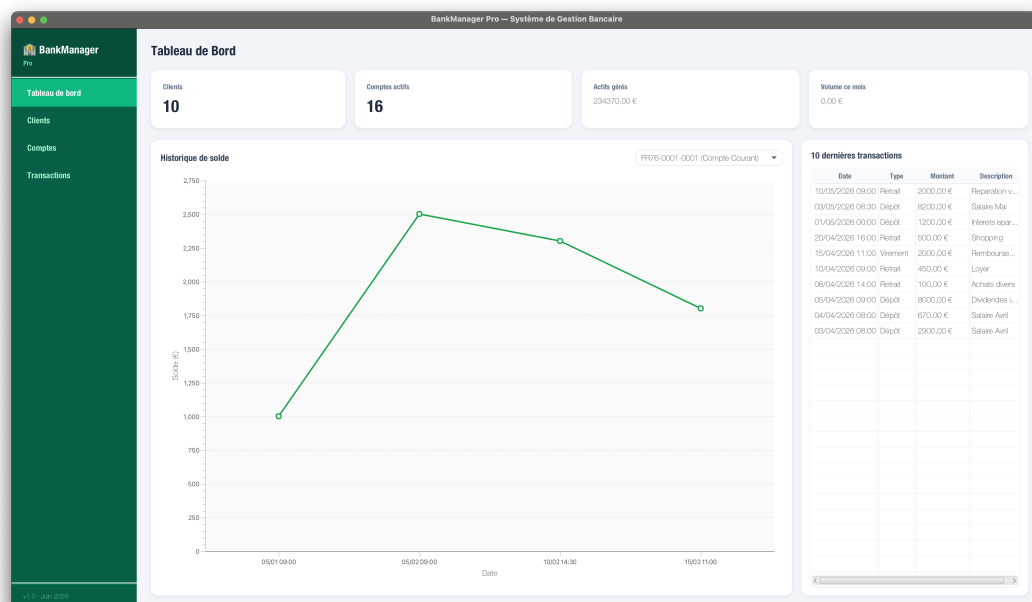


Figure 1 : Tableau de bord avec 10 clients, 16 comptes et le graphique d'historique de solde

4.2 Gestion des Clients

La section Clients permet de voir tous les clients enregistrés dans un tableau. On peut **ajouter** un nouveau client, **modifier** ses informations, ou le **supprimer**. La suppression n'est possible que si le client n'a plus de comptes associés (sinon l'application refuse et affiche un message d'erreur).

Il y a aussi une barre de recherche en haut à droite. On peut taper un prénom, un nom ou un email, et la liste se filtre en temps réel sans requête supplémentaire à la base de données (j'utilise une **FilteredList** de JavaFX).

ID	Prénom	Nom	Email	Téléphone	Inscription
4	Mohamed	Al Omani	mohamed.alomani@email.com	0061234567	14/09/2026
5	Fatima	Bernali	fatima.bernal@email.com	0072345678	14/09/2026
8	Abderrahman	Bensakim	abderrahman.bensakim@email.com	0035678901	14/09/2026
3	Claire	Bernard	claire.bernard@email.com	0055443322	14/09/2026
2	Bob	Dupont	bob.dupont@email.com	0098765432	14/09/2026
6	Youssef	Idrissi	youssef.idrissi@email.com	0083456789	14/09/2026
9	Khadja	Mansouri	khadja.mansouri@email.com	0026789012	14/09/2026
1	Alice	Martin	alice.martin@email.com	0012345678	14/09/2026
10	Karim	Wazzani	karim.wazzani@email.com	0017890123	14/09/2026
7	Amina	Zani	amina.zani@email.com	0094567890	14/09/2026

Figure 2 : Liste des clients avec les noms marocains et français

Quand on clique sur « **+ Nouveau client** », une boîte de dialogue s'ouvre avec les champs à remplir. Les champs Prénom, Nom et Email sont obligatoires. Si on essaie de valider sans les remplir, l'application affiche une erreur et ne ferme pas la fenêtre.

Figure 3 : Formulaire d'ajout d'un nouveau client

4.3 Gestion des Comptes

La section Comptes liste tous les comptes bancaires avec leurs informations : numéro de compte, client propriétaire, type de compte, solde actuel, limite de découvert, taux d'intérêt et statut (actif/inactif).

Il y a trois types de comptes : **Compte Courant**, **Compte Épargne** et **Crédit**. Les soldes sont affichés en vert quand ils sont positifs, et en rouge quand ils sont négatifs (découvert).

On peut filtrer les comptes par client ou par type de compte grâce aux menus déroulants en haut. Il y a aussi un bouton « **Activer/Désactiver** » pour bloquer un compte sans le supprimer.

ID	N° Compte	Client	Type	Solde	Découvert	Taux	Statut	Ouverture
1	FR76-0001-0001	Alice Martin	Compte Courant	3 500,00 €	500,00 €	0.00%	Actif	14/06/2026
2	FR76-0001-0002	Alice Martin	Compte Épargne	12 000,00 €	0,00 €	2.50%	Actif	14/06/2026
3	FR76-0002-0001	Bob Dupont	Compte Courant	850,00 €	200,00 €	0.00%	Actif	14/06/2026
4	FR76-0002-0002	Bob Dupont	Crédit	0,00 €	3000,00 €	15.00%	Actif	14/06/2026
5	FR76-0003-0001	Claire Bernard	Compte Épargne	45 000,00 €	0,00 €	3.00%	Actif	14/06/2026
6	FR76-0004-0001	Mohamed Al-Ouari	Compte Courant	8 200,00 €	1000,00 €	0.00%	Actif	14/06/2026
7	FR76-0004-0002	Mohamed Al-Ouari	Compte Épargne	25 000,00 €	0,00 €	3.50%	Actif	14/06/2026
8	FR76-0005-0001	Fatima Benali	Compte Courant	1 350,00 €	300,00 €	0.00%	Actif	14/06/2026
9	FR76-0006-0001	Youssef Idissi	Compte Épargne	62 000,00 €	0,00 €	4.00%	Actif	14/06/2026
10	FR76-0006-0002	Youssef Idissi	Crédit	0,00 €	5000,00 €	12.50%	Actif	14/06/2026
11	FR76-0007-0001	Amina Ziani	Compte Courant	2 900,00 €	500,00 €	0.00%	Actif	14/06/2026
12	FR76-0008-0001	Abderrahman Bensalem	Compte Courant	15 400,00 €	2000,00 €	0.00%	Actif	14/06/2026
13	FR76-0008-0002	Abderrahman Bensalem	Compte Épargne	38 000,00 €	0,00 €	3.00%	Actif	14/06/2026
14	FR76-0009-0001	Khadja Mersoui	Compte Courant	670,00 €	200,00 €	0.00%	Actif	14/06/2026
15	FR76-0010-0001	Karim Wazzani	Compte Épargne	19 500,00 €	0,00 €	2.75%	Actif	14/06/2026
16	FR76-0010-0002	Karim Wazzani	Crédit	0,00 €	4000,00 €	14.00%	Actif	14/06/2026

Figure 4 : Liste des 16 comptes avec soldes colorés en vert et statut

Pour créer un nouveau compte, on choisit un client dans la liste déroulante, le type de compte, et optionnellement on configure la limite de découvert et le taux d'intérêt. Le numéro de compte est généré automatiquement (format : FR76-XXXX-XXXX).

Figure 5 : Dialogue de création d'un nouveau compte bancaire

4.4 Gestion des Transactions

C'est la partie la plus importante techniquement. La section Transactions permet d'effectuer trois types d'opérations :

- **Dépôt** : créditer un compte depuis l'extérieur (salaire, bonus...)
- **Retrait** : débiter un compte avec vérification du solde disponible
- **Virement** : transférer de l'argent d'un compte vers un autre

Chaque opération met à jour le solde du compte **en même temps** qu'elle enregistre la transaction, grâce à une transaction JDBC (setAutoCommit(false)). Si quelque chose échoue, tout est annulé automatiquement (rollback).

ID	Date	Type	Compte source	Compte cible	Montant	Description
29	10/05/2028 09:00	Retrait	FR76-0008-0001	Externe	2000,00 €	Reparation voiture
28	09/05/2028 08:30	Dépôt	Externe	FR76-0004-0001	8200,00 €	Salaires Mai
30	01/05/2028 09:00	Dépôt	Externe	FR76-0001-0002	1200,00 €	Interests epargne
27	20/04/2028 16:00	Retrait	FR76-0004-0001	Externe	500,00 €	Shopping
26	15/04/2028 11:00	Virement	FR76-0010-0001	FR76-0010-0002	2000,00 €	Remboursement carte
18	10/04/2028 09:00	Retrait	FR76-0007-0001	Externe	450,00 €	Loyer
23	06/04/2028 14:00	Retrait	FR76-0009-0001	Externe	100,00 €	Achats divers
21	05/04/2028 09:00	Dépôt	Externe	FR76-0008-0002	8000,00 €	Dividendes investissement
22	04/04/2028 06:00	Dépôt	Externe	FR76-0009-0001	670,00 €	Salaires Avril
17	03/04/2028 08:00	Dépôt	Externe	FR76-0007-0001	2900,00 €	Salaires Avril
7	01/04/2028 10:00	Dépôt	Externe	FR76-0001-0002	5000,00 €	Bonus annuel
25	25/03/2028 10:00	Retrait	FR76-0010-0001	Externe	1200,00 €	Remboursement petit
20	20/03/2028 10:30	Virement	FR76-0009-0001	FR76-0008-0002	5000,00 €	Virement epargne
16	15/03/2028 12:00	Virement	FR76-0006-0001	FR76-0007-0001	3000,00 €	Aide familiale
6	13/03/2028 16:00	Retrait	FR76-0002-0001	Externe	150,00 €	Courses
24	05/03/2028 09:00	Dépôt	Externe	FR76-0010-0001	4500,00 €	Salaires Mars
5	05/03/2028 09:00	Dépôt	Externe	FR76-0002-0001	3000,00 €	Salaires Mars
15	05/03/2028 08:30	Dépôt	Externe	FR76-0006-0001	10000,00 €	Salaires Mars
19	04/03/2028 08:00	Dépôt	Externe	FR76-0006-0001	15400,00 €	Salaires Mars
13	20/02/2028 11:30	Retrait	FR76-0005-0001	Externe	200,00 €	Facture electricite
11	18/02/2028 15:00	Retrait	FR76-0004-0002	Externe	800,00 €	Retrait DAB
4	15/02/2028 11:00	Virement	FR76-0001-0001	FR76-0001-0002	500,00 €	Virement epargne
3	10/02/2028 14:30	Retrait	FR76-0001-0001	Externe	200,00 €	Retrait DAB
2	05/02/2028 09:00	Dépôt	Externe	FR76-0001-0001	1500,00 €	Salaires Février
12	05/02/2028 08:00	Dépôt	Externe	FR76-0005-0001	13850,00 €	Salaires Février
10	01/02/2028 09:00	Dépôt	Externe	FR76-0004-0002	5000,00 €	Bonus annuel
9	20/01/2028 10:00	Virement	FR76-0004-0001	FR76-0004-0002	2000,00 €	Virement epargne
14	10/01/2028 09:00	Dépôt	Externe	FR76-0006-0001	15000,00 €	Depot initial

Figure 6 : Historique complet des 30 transactions avec couleurs par type

Les transactions sont colorées : vert pour les dépôts, rouge pour les retraits, bleu pour les virements. On peut filtrer par type, par compte, et par plage de dates.

Dépôt

Effectuer un dépôt sur un compte

Compte cible *

Montant (€) *

Description

Cancel Déposer

Virement

Effectuer un virement entre comptes

Compte source *

Compte cible *

Montant (€) *

Description

Cancel Virer

Figure 7 : Dialogue de dépôt (gauche) et de virement entre comptes (droite)

5. Comment ça marche techniquement

5.1 Le pattern DAO

Toutes les requêtes SQL passent par des classes DAO (Data Access Object). Par exemple, **ClientDAO** a des méthodes comme `insert()`, `findAll()`, `search()`, `update()` et `delete()`. Les contrôleurs JavaFX appellent ces méthodes sans jamais écrire de SQL directement.

■ ■ **Sécurité SQL** : Toutes les requêtes utilisent des `PreparedStatement` avec des paramètres (?), jamais de concaténation de chaînes. Ça évite les injections SQL.

5.2 La connexion base de données

DatabaseManager est un singleton qui gère une seule connexion JDBC partagée entre tous les DAOs. Au premier lancement, il lit le fichier **schema.sql** depuis les ressources et crée toutes les tables automatiquement (`CREATE TABLE IF NOT EXISTS`). La base de données seed data est aussi dans ce fichier.

5.3 Les transactions atomiques

Pour les opérations bancaires (dépôt, retrait, virement), j'utilise des transactions JDBC pour garantir que les deux opérations (mise à jour du solde + enregistrement de la transaction) se font ensemble :

```
conn.setAutoCommit(false);  
// 1. mettre à jour le solde du compte  
// 2. insérer la transaction  
conn.commit(); // ou conn.rollback() si erreur
```

Si une des deux étapes échoue (par exemple solde insuffisant), tout est annulé. Ça évite d'avoir un solde mis à jour sans transaction enregistrée, ou vice versa.

5.4 L'interface JavaFX

L'interface utilise une architecture en couches : un fichier **main.fxml** qui définit la barre latérale et la zone de contenu centrale. Quand on clique sur un bouton de navigation, le contrôleur principal charge le fichier FXML correspondant dans la zone centrale.

Les modèles utilisent les **JavaFX Properties** (`StringProperty`, `DoubleProperty`...) pour que les colonnes des `TableViews` se mettent à jour automatiquement sans code supplémentaire.

6. Conclusion

Ce projet m'a permis de mettre en pratique tout ce qu'on a vu dans le module Java : la programmation orientée objet, les interfaces graphiques avec JavaFX, la connexion à une base de données, et l'organisation du code.

La partie la plus difficile était de gérer les transactions bancaires de façon atomique : s'assurer que le solde et l'historique sont toujours cohérents. J'ai aussi passé du temps sur le CSS pour que l'interface ait l'air propre et professionnel.

La fonctionnalité qui m'a le plus intéressé à implémenter, c'est le graphique d'historique de solde dans le tableau de bord. Le principe de reconstruire l'évolution du solde en rejouant les transactions m'a semblé élégant par rapport à stocker des snapshots.

Si j'avais plus de temps, j'aurais aimé ajouter un système d'authentification pour les employés, et peut-être un export PDF des relevés de compte. Mais pour ce qu'on demandait (CRUD complet, interface JavaFX, base de données, architecture propre), je pense que l'application répond bien aux exigences.

■ **Récapitulatif** : Application 100% fonctionnelle avec 4 vues complètes, 10 clients et 30 transactions de test, architecture MVC + DAO, transactions JDBC atomiques, graphique d'historique interactif, et interface en vert émeraude.

Rapport réalisé dans le cadre du module Programmation Java | SUPMTI Rabat | Juin 2026